

Netflix Insights and Metrics

MILESTONE 1: Cleaning and Normalisation

Week 1: Cleaning

1. Dataset Overview

The dataset was loaded from `netflix_titles.csv` into a Pandas DataFrame.

The initial shape of the dataset is (8807 rows, 12 columns). It includes 8,807 titles with 12 attributes:

- **title**
 - **type** (Movie / TV Show)
 - **director**
 - **cast**
 - **country**
 - **date_added**
 - **release_year**
 - **rating**
 - **duration**
 - **listed_in** (genres)
 - **description**
-

1.1. Data Cleaning Steps (Pandas)

1.11 Duplicate Removal

- Checked duplicates using `df_read.duplicated().sum()`.
- Found **0 duplicates** and dropped them using:
`df_read = df_read.drop_duplicates()`
- The **shape of the dataset remained (8807, 12)**.

1.12 Missing Value Handling

- Identified missing values using `df_read.isnull().sum()`.
- Filled missing values in key fields with default indicators using the `.fillna()` function:
- Filled missing values in key fields:

- director → *Unknown*
- cast → *Not Available*
- country → *Unknown*
- date_added → *Not Available*
- rating → *Not Rated*
- duration → *Unknown*
- Dropped rows missing critical fields: title, type.

1.13 Standardization

- Trimmed whitespaces and normalized formatting:
 - duration cleaned (stripped, converted to Title Case).

```
df_read['duration'] = df_read['duration'].str.strip().str.title()
```

- cast standardized by stripping whitespace.

```
df_read['cast'] = df_read['cast'].str.strip()
```

1.14 DataFrame Consolidation

- Saved the cleaned dataset as `df_clean`.
- The **final shape of the cleaned dataset (df_clean) is (8807 rows, 12 columns)**.
- Exported to CSV as `netflix_cleaned.csv` for downstream use with:

```
df_clean.to_csv("netflix_cleaned.csv", index=False)
```

1.2. Key Insights

All key insights were generated using the `.value_counts()` method in Pandas.

Content Distribution (Movies vs. TV Shows)

- Counted how many entries were **Movies** vs. **TV Shows**, showing Netflix's balance of formats using

```
type_distribution = df_clean['type'].value_counts()
```

- The analysis found: **6,131 Movies (69.6%)** and **2,676 TV Shows (30.4%)**.

Top Directors

- Extracted the Top 10 directors using:

```
top_directors = df_clean['director'].value_counts().head(10)
```

- Top directors included **Rajiv Chilaka (19 titles)**, **Raúl Campos, Jan Suter (18 titles)**, and **Marcus Raboy (16 titles)**.

Geographical Spread

- Extracted the Top 10 countries with most content using:

```
top_countries = df_clean['country'].value_counts().head(10)
```

- The top countries are the **United States (2,818 titles)**, **India (972 titles)**, and the **United Kingdom (419 titles)**.

Ratings

- Listed the **Top 10 most common ratings**, highlighting Netflix's most frequent audience classifications using:

```
top_ratings = df_clean['rating'].value_counts().head(10)
```

- The most frequent audience classifications are **TV-MA (3,207 titles)**, **TV-14 (2,160 titles)**, and **TV-PG (863 titles)**.

1.3. Potential Applications

- **Content Strategy** → Use director and country-level insights to plan future acquisitions.
 - **Genre & Rating Focus** → Explore dominant categories for personalized recommendations.
 - **Regional Growth** → Understand high-content countries to strengthen global strategy.
 - **Recommendation Systems** → Combine attributes like type, country, and rating to build content filters.
-

Week 2 Insights: Normalization

2. Data Normalization

To prepare the dataset for machine learning, categorical data was encoded into numerical formats, and key features were normalized. This process was performed on a copy of the cleaned DataFrame (df_clean).

2.1 Label Encoding for 'rating'

- Converted the categorical rating column into numerical labels using LabelEncoder.
- This assigns a unique integer to each rating category (e.g., TV-MA, TV-14), creating the new rating_label column.

```
label_encoder = LabelEncoder()
```

```
df_normalised['rating_label']=label_encoder.fit_transform(df_normalised['rating'].astype(str))
```

2.2 One-Hot Encoding for 'type'

- Applied OneHotEncoder to the type column to create binary features for 'Movie' and 'TV Show'.
- This avoids implying an incorrect ordinal relationship between the two categories.
- The resulting columns were merged back into the main DataFrame.

```
onehot_encoder = OneHotEncoder(sparse_output=False, drop=None)
```

```
type_encoded = onehot_encoder.fit_transform(df_normalised[['type']])
```

```
df_type_onehot = pd.DataFrame(type_encoded,
```

```
columns=onehot_encoder.get_feature_names_out(['type'])) df_normalised =  
pd.concat([df_normalised, df_type_onehot], axis=1)
```

2.3 Ordinal Encoding for 'country'

- Transformed the country column into ranked numerical values using OrdinalEncoder.

- The encoding order was based on the frequency of content from each country (`value_counts()`).
- A new `country_ordinal` column was added to the DataFrame.

```
country_order = df_normalised['country'].value_counts().index.tolist()
```

```
ordinal_encoder = OrdinalEncoder(categories=[country_order])
```

```
df_normalised['country_ordinal']=ordinal_encoder.fit_transform(df_normalised[['country']])
```

2.4 Normalization (Min-Max Scaling)

- Applied `MinMaxScaler` to scale numerical features to a standard range between 0 and 1.
- This step is important for machine learning algorithms sensitive to the scale of input features.
- The following newly created columns were normalized:
 - **rating_label**
 - **country_ordinal**

```
scaler = MinMaxScaler() cols_to_normalize = ['rating_label', 'country_ordinal']
```

```
df_normalised[cols_to_normalize] = scaler.fit_transform(df_normalised[cols_to_normalize])
```

MILESTONE 2: EDA and Feature Engineering

Week 3: EDA

The Exploratory Data Analysis (EDA) was performed on the `netflix_cleaned.csv` DataFrame, which has a shape of **(8807 rows, 12 columns)**. The analysis focused on understanding content composition, temporal trends, and key descriptive statistics.

3.1. Data Quality Check

The cleaning process successfully handled most missing values, leaving only two columns with minor missing data that required further handling for temporal analysis:

- **Null Count:** `date_added` and the derived `year_added` each had **10 null values**.
- **Percentage of Nulls:** This represents a minimal **0.113546%** of the dataset.
- **Handling Code:** Missing values in the `year_added` column were filled with **0** for consistency:

```
df['year_added'] = df['year_added'].fillna(0)
```

3.2. Content Distribution Analysis

Content Type (Movies vs. TV Shows)

- **Insight:** Netflix's catalog is primarily composed of **Movies**, which account for over two-thirds of the content.
- **Distribution:** **Movies** make up **69.6%** (6,131 titles), while **TV Shows** make up **30.4%** (2,676 titles).

```
type_counts = df['type'].value_counts()
```

```
type_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90)
```

Rating Distribution

- **Insight:** The most frequent audience classifications are those aimed at mature and young adult audiences.

- **Top Ratings:** The three most common ratings are **TV-MA** (Mature Audiences), **TV-14** (Teens 14+), and **TV-PG** (Parental Guidance Suggested).

```
rating_counts = df['rating'].value_counts()
rating_counts.plot(kind='bar')
```

3.3. Temporal & Categorical Insights

Content Growth Over Time

- **Insight:** The number of titles added to Netflix saw its steepest increase and peaked around **2019-2020**, indicating a period of aggressive content acquisition before a potential slowdown.

```
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
df['year_added'] = df['date_added'].dt.year
content_growth = df.groupby('year_added').size()
```

3.31. Top Genres

- **Insight:** When separating the comma-delimited `listed_in` column, **Dramas**, **Comedies**, and **International Movies/TV Shows** emerge as the dominant genres across the platform.

```
genres = df['listed_in'].dropna().str.split(',').explode().str.strip()
genre_counts = genres.value_counts().head(15)
```

3.32. Top Countries Contributing Content

- **Insight:** Content is heavily skewed towards the **United States**, which is the clear leader in contributions.

```
countries = df['country'].dropna().str.split(',').explode().str.strip()
country_counts = countries.value_counts().head(15)
```

3.4. Derived Feature Insights

3.41. Movie Length Categories

- The duration column (e.g., "90 Min") was cleaned and categorized into three bins: **Short** (0-60 min), **Medium** (60-120 min), and **Long** (120+ min).
- **Insight:** The majority of movies fall into the **Medium** length category (60-120 minutes).

```

movies['duration_num'] = (
    movies['duration'].str.extract('(\d+)').astype(float)
)
movies['length_category'] = pd.cut(
    movies['duration_num'],
    bins=[0, 60, 120, float('inf')],
    labels=['Short', 'Medium', 'Long']
)

```

3.42. Original vs. Licensed Content Proxy

- **Feature Engineering:** A binary feature, `is_original`, was created as a proxy for Netflix Original content by checking if the word "Netflix" (case-insensitive) was present in the title.
- **Insight:** This proxy feature shows the distribution of titles that explicitly contain "Netflix" in the title versus other licensed content.

```

df['is_original'] = df['title'].str.contains('(?!i)netflix', na=False)
df['origin_label'] = df['is_original'].map({True: 'Original', False:
    'Licensed'})

```

BIVARIATE

3.5. Content Growth Over Time (Scatter Plot)

This plot shows the volume of new content (both Movies and TV Shows) added to Netflix each year, illustrating the platform's content acquisition strategy.

- **Date Conversion:** The `date_added` column was converted to datetime objects, and the year was extracted into `year_added`.
- **Grouping:** The data was **grouped by year_added** and the count of titles was calculated.
- **Visualization:** A **scatter plot** was used to visualize the trend

```

df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
df['year_added'] = df['date_added'].dt.year
content_growth = df.groupby('year_added').size().reset_index(name='count')
# ... plotting code ...

```

Key Insight

The chart clearly shows an **exponential growth** in content additions from around 2015, peaking sharply around **2019-2020**. This trend reflects Netflix's massive global expansion and investment in content during that period.

3.6. Movie Duration vs. Year Added (Seaborn Scatter Plot)

This plot filters for movies and explores the relationship between the year a movie was added and its running time, segmented by whether it's an "Original" or "Licensed" title.

- **Filtering & Duration Extraction:** The DataFrame was filtered for 'Movie', and the numerical duration (minutes) was extracted into `duration_num`.
- **Origin Label:** The `origin_label` column (Original/Licensed) was used for color coding. (This column was pre-calculated by checking for "Netflix" in the title).
- **Visualization:** A **Seaborn scatter plot** compares `year_added` (X-axis) against `duration_num` (Y-axis), colored by `origin_label`.

```
movies = df[df['type'] == 'Movie'].copy()
movies['duration_num'] = (
    movies['duration'].str.extract('(\d+)').astype(float)
)
# ...
sns.scatterplot(
    data=movies,
    x='year_added',
    y='duration_num',
    hue='origin_label',
    alpha=0.7
)
```

Key Insight

The distribution of **Licensed** movies (yellow/green in the plot) covers a wider range of durations and release years, while **Original** movies (blue/red) are concentrated in **recent years (post-2015)** and primarily cluster around the **90 to 110-minute** mark. This suggests Netflix's

internally branded movie production favors a standard, medium-length format.

3.7. Distribution of Movie Duration by Rating (Seaborn Histogram)

This stacked histogram visualizes the runtime distribution of movies and breaks down the total count by their MPAA/TV rating.

- **Data:** Uses the movies DataFrame and the duration_num column.
- **Visualization:** A Seaborn stacked histogram plots the movie duration distribution, with the bars segmented (stacked) by the rating category.

```
sns.histplot(  
    data=movies,  
    x='duration_num',  
    hue='rating',  
    multiple='stack',  
    palette='Set1',  
    bins=30  
)
```

Key Insight

The majority of movies fall between **80 and 120 minutes**. Within this central peak, **TV-MA** and **TV-14** ratings dominate the volume, consistent with the overall high frequency of these ratings. This plot is essential for determining if extreme durations (very short or very long films) are predominantly associated with a specific audience rating (e.g., whether shorter content is mostly TV-Y)

WEEK 4:FEATURE ENGINEERING

4. Derived Feature Insights

4.1. Movie Length Categories

- A unified feature, content_length_category, was created to capture content length for both movies and TV shows.
- **Movies:** Duration (from the duration column, e.g., "90 Min") was extracted numerically and categorized into:
 - Short (0–60 min)
 - Medium (60–120 min)
 - Long (120+ min)

- **TV Shows:** Number of seasons (from the duration column, e.g., "2 Seasons") was extracted and categorized into:
 - Single Season (1 season)
 - Multi Season (2 or more seasons)

Key Insight:

- Most movies fall into the Medium category (60–120 minutes).
- Most TV shows are Single Season, reflecting Netflix's content strategy favoring limited series or short-run acquisitions.

```
movies['length_category'] = pd.cut(
    movies['duration_num'],
    bins=[0, 60, 120, float('inf')],
    labels=['Short', 'Medium', 'Long']
)
tv_shows['season_category'] = pd.cut(
    tv_shows['seasons_num'],
    bins=[0, 1.5, float('inf')],
    labels=['Single Season', 'Multi Season']
)
```

4.2. Original vs. Licensed Content Proxy

- **Feature Engineering:** A binary feature, `origin_label`, was created as a proxy for Netflix Original content by checking if the word "Netflix" (case-insensitive) was present in the title.
- **Insight:** This proxy feature shows the distribution of titles that explicitly contain "Netflix" in the title versus other licensed content.

```
df['origin_label'] = df['is_original'].map({True: 'Original', False:
    'Licensed'})
```

4.3. Content Age Categories Analysis

This analysis derives the age of each title (as of the hypothetical year 2025) and groups them into specific age ranges to understand the freshness of the catalog.

1. **Age Calculation:** The age of the content was calculated by subtracting the `release_year` from 2025.
2. **Categorization:** The resulting `content_age` was grouped into four bins: '0–5 yrs', '6–10 yrs', '11–20 yrs', and '20+ yrs'.

Key Insight

The majority of the content falls into the 6–10 yrs range, indicating a significant volume of acquired content was originally released between 2015 and 2019.

4.4. Title Length Categories Analysis

This analysis derives a new feature based on the character length of the content titles and categorizes them into four defined groups.

1. Length Calculation: The title length was calculated for every entry.
2. Categorization: Title lengths were categorized using the following bins:
 - Short: 0–10 characters
 - Medium: 11–20 characters
 - Long: 21–40 characters
 - Very Long: 40+ characters

Key Insight

The Medium (11–20 characters) category holds the largest number of titles (3,773), followed by Short (2,366) and Long (2,301). Very long titles are relatively rare. This distribution indicates that most content is branded with titles concise enough to be easily recognized, while still being descriptive.

MILESTONE 3: Modeling & Advanced Analysis

Week 5 - Modeling

5. Unsupervised Learning: K-Means Clustering

K-Means clustering was performed to identify natural groupings of content based on key features, with the goal of creating content segments (clusters).

5.1.1 Clustering Methodology

The model was configured to create 5 clusters (`n_clusters=5`). The following features were prepared and scaled for the model:

- **Genre:** The multi-value `listed_in` column was transformed into binary features using `MultiLabelBinarizer`.
- **Duration:** The duration column was converted to a single numeric feature. For TV Shows, seasons were approximated as minutes (e.g., "1 Season" ~ 60 minutes) to create a numerical scale comparable to movies (e.g., "90 Min").
- **Rating:** The categorical rating field was converted into numerical labels using simple ordinal mapping.
- **Scaling:** All combined features (X) were standardized using `StandardScaler()` to ensure that the magnitude of features (like duration) did not unfairly dominate the clustering process.

5.1.2. Cluster Distribution Results

The K-Means algorithm grouped the titles into 5 segments with the following distribution:

Cluster	Count
0	509
1	913
2	5991
3	29
4	1265

Key Insight

Cluster 2 is overwhelmingly the **largest segment**, containing the majority of the dataset (5,991 titles). This suggests a vast amount of content shares similar attributes, likely dominated by standard movie lengths and common international or drama genres, while other clusters represent distinct niches (like Clust

5.2. Supervised Learning: Random Forest Classification

A Random Forest Classifier was trained to predict the content type (**Movie** or **TV Show**) based on features derived from the EDA steps.

5.2.1. Classification Methodology

- **Target Variable (y):** type (Mapped to 0 for 'Movie', 1 for 'TV Show').
- **Features (X):** duration, content_age_years, primary_genre, and rating.
- **Feature Preparation:** Categorical features (primary_genre, rating) were encoded using **label encoding**, and duration was converted to a floating-point number.
- **Model:** A RandomForestClassifier was trained on a 80/20 train/test split of the data.

5.2.2. Model Performance

The Random Forest model achieved near-perfect separation between Movies and TV Shows.

	Precision	Recall	F1-Score
Movie	1.00	1.00	1.00
TV Show	1.00	1.00	1.00
Accuracy			1.00

Key Insights

The model's perfect scores (1.00 for all metrics) strongly suggest that the features selected contain a **clear separation** between the two classes. Specifically, the duration field, which retains the minutes/seasons distinction, allows the classifier to perfectly discriminate between movies and TV shows.

5.2.3. Feature Importance

The importance of each feature in determining the content type was evaluated by the Random Forest model:

Feature	Importance
duration	0.851
primary_genre	0.121
rating	0.026
content_age_years	0.002

Key Insight: The **duration** feature dominates the prediction with an importance score of **0.851**. This is expected, as the numerical encoding of duration (minutes vs. number of seasons converted to minutes) strongly distinguishes the Movie category from the TV Show category, confirming that **content duration is the single most defining attribute** for classifying a Netflix title as a Movie or a TV Show.